A Sequential Decision-Making Framework for a Two-Player Resource-and-Combat Game

ASEN 5264 — Decision Making Under Uncertainty

Michael Sola
University of Colorado Boulder

Jeremiah Hoffman
University of Colorado Boulder

I. Introduction

This project develops a two-player strategy game in which each player manages workers, marines, and resources with the goal of eliminating all enemy marines. The game is formalized as a Markov Decision Process and solved using methods from the course, with increasing model complexity across three tiers of completion. The first tier utilizes value iteration to solve for an optimal policy against a deterministic policy of the adversary and Monte Carlo Tree Search (MCTS) to evaluate online planning effectiveness. Reinforcement learning is also applied, specifically Q-learning and Deep Q-Network (DQN) learning to learn against this optimal policy. The second tier extends the problem formulation in two branches. First, it is extended to a Partially Observable Markov Decision Process (POMDP), where the player does not have full state knowledge, which is approximately solved with both the Q-Value MDP approximation (QMDP) and Partially Observable Monte Carlo Planning (POMCP). Second, it is simulated as a Markov Game (MG), where both players have dynamic gameplay policies. The Markov Game is solved using fictitious self-play, in which each player iteratively best-responds against the running average of all past opponent policies, guaranteeing convergence to a Nash equilibrium in this zero-sum setting. The third tier would combine the MG and POMDP into a Partially Observable Stochastic Game (POSG). Among MDP solvers, value iteration achieved the highest win rate but at the cost of full state-space enumeration; DQN approached comparable performance with significantly better scalability. Under partial observability, QMDP proved robust to limited opponent state information but brittle to opponent policy mismatch, where POMCP's online planning maintained stronger performance by avoiding reliance on a precomputed value function. summarize MG and POSG results

II. Background and Related Work

The main motivation behind this project was to simulate deterrence as a POSG. However, the problem needed to be dramatically simplified to be able to be completed in the time alloted. Adjacent to the concept of deterrence is conflict simulation. Real time strategy (RTS) games are the most analogous to conflict simulation and is a platform that can be integrated into a reinforcement learning platform. RTS games possess many attributes that make reinforcement learning difficult: partial observability, large state and action spaces, rapid response times, and evolving strategies to name a few. In the field of reinforcement learning, AlphaStar is considered one of the biggest achievements cite alphastar where it was taught to play Starcrat II (SC2). AlphaStar first used supervised learning to produce an initial policy by learning on previously recorded matches. AlphaStar used a variant of actor-critic as its main learning algorithm. On the multi-agent side, AlphaStarr used fictious self play to compute the best responses against a variety of opponents.

III. Problem Formulation

A. State Space

The state for each player $i \in \{1, 2\}$ is (w_i, m_i, r_i), representing workers, marines, and resources. The joint state is $s = (w_1, m_1, r_1, w_2, m_2, r_2)$, plus a terminal state s_{term}. Both players start at $w = m = r = 1$. Resources increase by w_i each turn. Workers, marines, and resources for each player are limited to $w_{\max} = m_{\max} = r_{\max} = 10$, yielding approximately 3.5M total states.

B. Action Space

Each player selects from the following action set each turn:

$$\mathcal{A} = \{\texttt{train worker}, \texttt{train marine}, \texttt{attack}\}$$

C. Transition Model

The table below summarizes the transition model. If either player has $m_i = 0$, the game transitions to s_{term} with probability 1 regardless of action.

TABLE I
Transition Model

Action	Condition	Next State	Prob.
train worker	$r \geq 1$	$w' = w + 1,\ r' = r - 1$	0.9
train marine	$r \geq 1$	$m' = m + r,\ r' = 0$	0.9
attack	always	$m'_A = m_A - k_A,\ m'_B = m_B - k_B$	$P(k_A, k_B)$
terminal	$m_i = 0$	s_{term}	1.0

where $k_A \sim \text{Binomial}(m_B, p_B)$, $k_B \sim \text{Binomial}(m_A, p_A)$, and $P(k_A, k_B)$ is their product under independence.

D. Reward

$R = 1$ upon reaching s_{term} as the winner; $R = 0$ otherwise.

IV. Solution Approach

A. Tier 1: Value Iteration on the Fully Observable MDP

The game is formalized as a fully observable MDP with pre-computed transition matrices T_a for each Player 1 action a. Player 2's default action policy, `alternating_training_attack`, alternates between training marines and workers unless both are saturated ($m_2 = w_2 = 10$), in which case the attack action is selected. This MDP is solved via value iteration (Algorithm 1) to obtain an optimal policy π^*, iterating the Bellman update

$$U'(s) = \max_{a \in \mathcal{A}} \left(R(s,a) + \gamma \sum_{s' \in \mathcal{S}} T(s' \mid s,a)\, U(s') \right) \tag{1}$$

until $\|U - U'\|_\infty < \varepsilon$. The greedy policy is then extracted as $\pi^*(s) = \arg\max_a \big(R(s,a) + \gamma\, \mathbb{E}[U^*(s')]\big)$.

Algorithm 1 Value Iteration

Given: MDP $(\mathcal{S}, \mathcal{A}, R, T, \gamma)$, tolerance ε

Initialize U, U' (differently)
repeat
  $U \leftarrow U'$
  for each $s \in \mathcal{S}$ **do**
    $U'(s) \leftarrow \max_{a \in \mathcal{A}} \left(R(s,a) + \gamma \sum_{s' \in \mathcal{S}} T(s' \mid s,a)\, U(s') \right)$
  end for
until $\|U - U'\|_\infty \le \varepsilon$
return U'

B. Tier 1: Online Planning with MCTS

Since value iteration requires full-state enumeration, Monte Carlo Tree Search (MCTS/UCT, Algorithm 2) is also implemented as an online planner. From each state encountered during gameplay, MCTS runs $N = 1000$ simulations, each of which selects actions via the UCB criterion $Q(s,a) + c\sqrt{\ln N(s)/N(s,a)}$ with $c = \sqrt{2}$, expands the tree at leaf nodes, estimates value via a random rollout to depth 50, and backs up the discounted return. The agent then acts greedily with respect to the resulting Q estimates.

C. Tier 1: Reinforcement Learning: Q-Learning and DQN

Two reinforcement learning methods are applied. Q-learning (Algorithm 3) maintains a tabular estimate $Q(s,a)$ updated via the TD error at each step, with $\alpha = 0.1$, $\gamma = 0.95$, and ε decaying linearly from 0.2 to 0.05.

DQN (Algorithm 4) replaces the Q-table with a neural network $Q_\theta(s,a)$, enabling generalization to larger state spaces. The network maps the 7-dimensional state to a 256-unit hidden layer (ReLU) followed by a 3-element action output. Training uses a circular replay buffer (capacity 10^5), mini-batches of 50, target-network updates every 50 steps, and Adam with learning rate 5×10^{-4}. Exploration decays exponentially from $\varepsilon_0 = 0.5$ to $\varepsilon_{\min} = 0.01$.

Algorithm 2 Monte Carlo Tree Search (MCTS/UCT)

function MCTS(s)
  for $k = 1, \ldots, m$ **do**
    SIMULATE(s)
  end for
  return $\arg\max_{a \in \mathcal{A}} Q[(s,a)]$
end function

function SIMULATE(s, d)
  if $d \le 0$ **then**
    return $U(s)$ ▷ value estimate / rollout
  end if
  if s not yet in tree **then**
    for each $a \in \mathcal{A}$ **do**
      $N[(s,a)] \leftarrow 0$; $Q[(s,a)] \leftarrow 0$
    end for
    return $U(s)$
  end if
  $a \leftarrow \arg\max_a \left[Q[(s,a)] + c\sqrt{\dfrac{\ln N(s)}{N(s,a)}} \right]$
  $s', r \leftarrow \mathrm{TR}(s,a)$
  $q \leftarrow r + \gamma \cdot$ SIMULATE(s', $d-1$)
  $N[(s,a)] \leftarrow N[(s,a)] + 1$
  $Q[(s,a)] \leftarrow Q[(s,a)] + \dfrac{q - Q[(s,a)]}{N[(s,a)]}$
  return q
end function

Algorithm 3 Q-Learning

$Q(s,a) \leftarrow 0$ for all (s,a); $s \leftarrow s_0$
loop
  $a \leftarrow \begin{cases} \arg\max_a Q(s,a) & \text{w.p. } 1-\varepsilon \\ \mathrm{rand}(\mathcal{A}) & \text{otherwise} \end{cases}$
  $r \leftarrow$ `act!`(*env*, a); $s' \leftarrow$ `observe`(*env*)
  $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
  $s \leftarrow s'$
end loop

Algorithm 4 Deep Q-Network (DQN)

Initialize Q_θ; target network $Q_{\theta'} \leftarrow Q_\theta$; replay buffer $\mathcal{D} \leftarrow \emptyset$
loop
  $a \leftarrow \begin{cases} \arg\max_a Q_\theta(s,a) & \text{w.p. } 1-\varepsilon \\ \mathrm{rand}(\mathcal{A}) & \text{otherwise} \end{cases}$
  $r \leftarrow$ `act!`(*env*, a); $s' \leftarrow$ `observe`(*env*); store (s,a,r,s') in $\mathcal{D}$
  Sample mini-batch $\mathcal{B}$ from $\mathcal{D}$
  $\theta \leftarrow \theta - \alpha \nabla_\theta \frac{1}{|\mathcal{B}|} \sum_{(s_j,a_j,r_j,s'_j) \in \mathcal{B}} \big(r_j + \gamma \max_{a'} Q_{\theta'}(s'_j, a') - Q_\theta(s_j, a_j)\big)^2$
  Periodically: $\theta' \leftarrow \theta$ ▷ sync target network
  $s \leftarrow s'$
end loop

D. Tier 2: Partial Observability and QMDP

The game model is extended to a POMDP where Player 1 observes its own personnel and resources exactly but receives only a discretized observation of Player 2's personnel with granularity n. QMDP approximates the POMDP value function by assuming full observability after one step, yielding

$$Q_{\text{QMDP}}(b,a) = b^{\top}(R + \gamma T_a V^*), \qquad \pi_{\text{QMDP}}(b) = \arg\max_a Q_{\text{QMDP}}(b,a) \tag{2}$$

E. Tier 2: POMCP

While QMDP is simple to implement, it assumes next-step full observability and requires a full-state value function. POMCP (Algorithm 5) instead combines MCTS with a particle-based belief, indexing the search tree by action–observation history h rather than state. Each call draws 200 particles from the current belief, runs 5000 simulations using UCB, and performs random rollouts to depth 20 from leaf nodes.

Algorithm 5 POMCP

function POMCP(b)
  for $k = 1, \ldots, m$ **do**
    $s \sim b$ ▷ sample state from belief
    SIMULATE(s, h_0, d)
  end for
  return $\arg\max_{a \in \mathcal{A}} Q[(h_0, a)]$
end function

function SIMULATE(s, h, d)
  if $d = 0$ **then**
    return 0
  end if
  if $h \notin$ tree **then**
    for each $a \in \mathcal{A}$ **do**
      $N[(h,a)] \leftarrow 0; \quad Q[(h,a)] \leftarrow 0$
    end for
    return ROLLOUT(s, d) ▷ expansion + rollout
  end if
  $a \leftarrow \arg\max_a \left[Q[(h,a)] + c\sqrt{\dfrac{\ln N(h)}{N(h,a)}} \right]$
  $s', o, r \leftarrow G(s,a)$ ▷ generative model sample
  $q \leftarrow r + \gamma \cdot$ SIMULATE(s', $h \cdot ao$, $d-1$)
  $N[(h,a)] \leftarrow N[(h,a)] + 1$
  $Q[(h,a)] \leftarrow Q[(h,a)] + \dfrac{q - Q[(h,a)]}{N[(h,a)]}$
  return q
end function

F. Tier 2: Markov Game

All previous approaches fixed the opponent's policy, reducing the game to a single-agent MDP. A Markov game extends the MDP framework to the multi-agent setting by defining a joint action space and per-player reward functions. Two solution methods are applied: alternating best response (Algorithm 6) and fictitious self-play (Algorithm 7).

In ABR each player in turn freezes the opponent's current policy, reduces the MG to a single-agent MDP, and solves for a best response via value iteration. This can oscillate without converging since each player reacts only to the opponent's most recent policy. Fictitious self-play corrects this by maintaining a running average of each player's past best responses as a mixed strategy $\sigma^i(a \mid s)$, represented via action-count arrays $n^i[s,a]$ incremented after each solve. Each player best-responds against the opponent's empirical average σ^{-i}. In finite two-player zero-sum games, fictitious play is guaranteed to converge to a Nash equilibrium.

The combined transition and reward for Player 1 are marginalized over Player 2's mixed strategy:

$$T'(s' \mid s, a^1) = \sum_{a^2} T(s' \mid s, a^1, a^2)\, \sigma^2(a^2 \mid s), \tag{3}$$

$$R'(s, a^1) = \sum_{a^2} R(s, a^1, a^2)\, \sigma^2(a^2 \mid s), \tag{4}$$

and Player 1 runs value iteration against this marginalized MDP. The same procedure applies symmetrically for Player 2 using an inverted-state representation.

Algorithm 6 Alternating Best Response

Initialize policies π_0^1, π_0^2 arbitrarily
for iteration $k = 1, 2, \ldots, K$ **do**
  Fix π_{k-1}^2; reduce MG to single-agent MDP for P1
  $\pi_k^1 \leftarrow \mathrm{VI}\big(\mathrm{env}_{P1}(\pi_{k-1}^2)\big)$ ▷ P1 best response
  Fix π_k^1; reduce MG to single-agent MDP for P2
  $\pi_k^2 \leftarrow \mathrm{VI}\big(\mathrm{env}_{P2}(\pi_k^1)\big)$ ▷ P2 best response
end for
return $\pi_K^1,\ \pi_K^2$

Algorithm 7 Fictitious Self-Play

Initialize $n^1[s,a] \leftarrow 0$; seed n^2 from initial P2 policy
for iteration $k = 1, 2, \ldots, K$ **do**
  $\sigma^2 \leftarrow \mathrm{normalize}(n^2)$ ▷ P2 empirical average policy
  $\pi_k^1 \leftarrow \mathrm{VI}\big(\mathrm{env}_{P1}(\sigma^2)\big)$ ▷ P1 best response to σ^2
  $n^1[s,\ \pi_k^1(s)] \mathrel{+}= 1$ for all $s \in \mathcal{S}$
  $\sigma^1 \leftarrow \mathrm{normalize}(n^1)$ ▷ P1 empirical average policy
  $\pi_k^2 \leftarrow \mathrm{VI}\big(\mathrm{env}_{P2}(\sigma^1)\big)$ ▷ P2 best response to σ^1
  $n^2[s,\ \pi_k^2(s)] \mathrel{+}= 1$ for all $s \in \mathcal{S}$
end for
return $\sigma^1 \leftarrow \mathrm{normalize}(n^1), \quad \sigma^2 \leftarrow \mathrm{normalize}(n^2)$

G. Tier 3: Partially Observable Stochastic Game

The full formulation combines partial observability with simultaneous multi-agent interaction. Each player maintains a belief over the joint state and reasons about opponent behavior under uncertainty. Solution methods may include multi-agent reinforcement learning or approximate dynamic programming techniques.

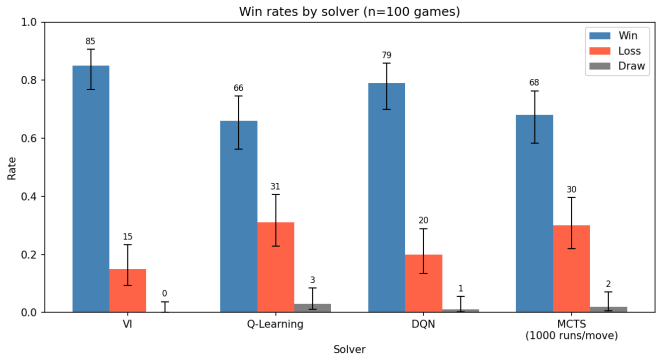

Fig. 1. MDP Solver Performance vs. `alternating_training_attack`. Error bars show 95% Wilson confidence intervals.

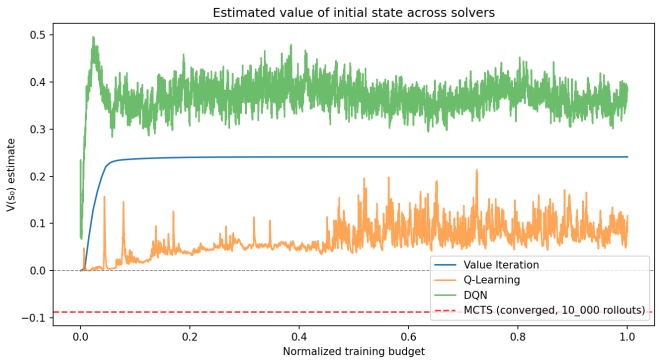

Fig. 2. Initial-state Value Estimate vs. normalized training/planning effort for each MDP solver

V. Results

A. Tier 1: MDP Solver Performance

All MDP solvers were evaluated from the Player 1 perspective against a fixed Player 2 policy using the following parameters to define the game environment and solver setup:

- Initial State: Workers = Marines = Resources = 1
- Player 2 policy: `alternating_training_attack`
- Number of episodes: 10,000
- Number of games to evaluate win rate: 100
- Discount factor: 0.95

As expected, value iteration achieved the best performance, since it converges to the optimal value function, from which a greedy policy can be directly extracted. However, this performance comes at the cost of additional computation time, since the value iteration solution method iterates over the entire state space, where approximate solvers need only a sufficient sample of states. For the current state dimensions, this is very tractable, converging in approximately 30 seconds, but future increases in complexity - higher troop or resource limits, more types of personnel, more action options, etc. - can increase the state space size exponentially, and value iteration would quickly become intractable as it scales with

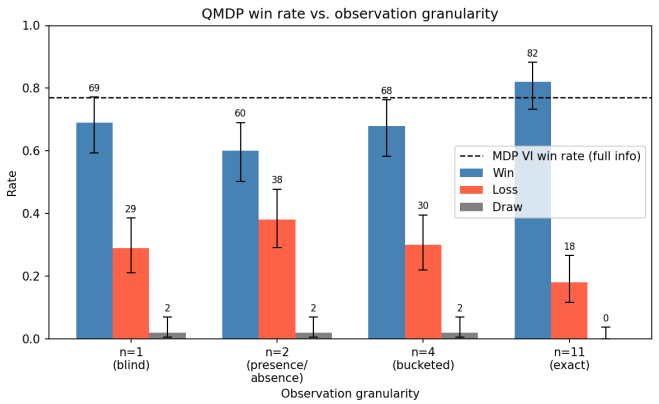

Fig. 3. QMDP Win Rate vs. Observation Granularity (n). Error bars show 95% Wilson confidence intervals.

$|\mathcal{S}|^2$. Q-learning scales better than value iteration ($|\mathcal{S}|$), but this could still become intractable, and as seen in Figure 1 Q-learning shows the worst performance of the implemented solvers. DQN and MCTS both attempt to minimize this curse of dimensionality: MCTS by exploring only a tree of reachable states, which would scale well to large state spaces, though it trades quality for this scalability, and may require significantly increased run counts in larger reachable state spaces to achieve acceptable performance. Since DQN approximates the value function with a neural network, it can generalize to unvisited states, and for a well-behaved value function can understand a very large state space without much added computational cost. This scalability appears to come at a much lower cost than MCTS for this problem, as DQN performs nearly as well as value iteration, making it a prime contender for use in future problem evolutions.

In Figure 2, the estimation errors from the other three solvers can be seen more directly. Q-learning and MCTS both underestimate the value of the initial state provided by value iteration, MCTS quite significantly, and DQN overestimates it. In future work, any chosen solver - likely DQN, based on its current performance - could be modified to account for some of this error, e.g., by implementing double Q-learning to remove maximization bias.

B. Tier 2: POMDP Observability

To explore the impacts of limited observation of the adversary's state, the QMDP solver was used in scenarios with varying levels of observability (no observability, presence or absence of personnel, bucketed estimate of personnel, and full observability). Each scenario was evaluated using the following parameters:

- Player 2 policy: `alternating_training_attack`
- Number of games: 100
- Step limit: 500

As expected, QMDP achieved the best win rate in the exact observation scenario, decreasing the observability of

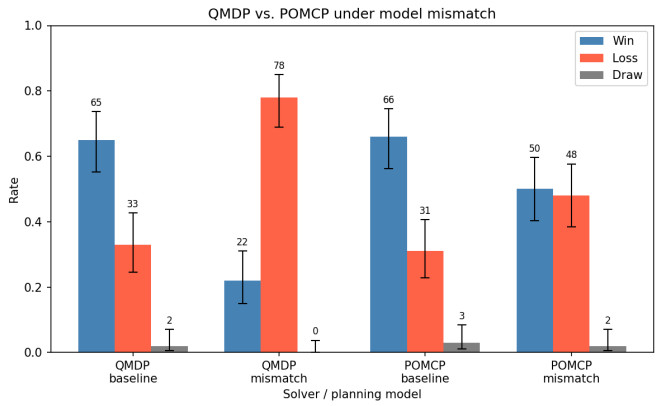

Fig. 4. Performance Under P2 Model Mismatch (planner assumes uniform-random P2; simulation uses `alternating_training_attack`). Error bars show 95% confidence intervals.

the opponent's state only decreased performance by a small margin. The performance for each of the bucketed, presence/absence, and blind scenarios is very similar, with the confidence intervals mostly overlapping. This indicates that knowledge of the adversary's population status is minimally important for winning the game. Stated another way, the best policy for Player 1 does not vary significantly based on Player 2's state, and QMDP's single-step lookahead is sufficiently capable for this POMDP formulation, achieving close to the MDP win rate even when observability is limited.

C. Tier 2: Model Mismatch and POMCP

In order to determine the effect of other sources of uncertainty, an additional scenario was constructed where the solver has no knowledge of Player 2's policy: uncertainty in this scenario comes not just from a partially observable state, but also from ambiguity over what actions the opponent will play from any given state. This was effectuated for the QMDP solver by building a value function from a uniform random Player 2 policy, but allowing Player 2 to instead play the `alternating_training_attack` policy in subsequent gameplay. Following an observed drop in QMDP's effectiveness under this uncertainty, the POMCP solver was also implemented to compare performance under the same conditions. The solver and gameplay parameters from the observation granularity investigation were reused, but here the enemy state remained fully unobservable (`n_obs_levels=1`).

The introduction of policy uncertainty significantly decreases QMDP's win rate, indicating that this POMDP solution method is brittle to an inaccurate value function - unsurprisingly for an offline planner. POMCP's results, as an online planner, are markedly different: it slightly outperforms QMDP in the baseline case (within the margin of error), but takes significantly longer to run - approximately 3x QMDP's runtime - as its Monte Carlo-based estimate of the value of a state must be built at each step. POMCP, however, can build a reasonably accurate value estimate regardless of the prior knowledge QMDP relies on, and its higher performance in the "mismatch" scenario demonstrates this robustness.

D. Markov Game

insert convergence plot insert game results

VI. Conclusion

This work formalized a two-player resource-and-combat game as an MDP and evaluated four solvers under full and partial observability. Value iteration produced the highest win rate (58%) against the fixed opponent policy, while DQN achieved near-comparable performance (50%) with substantially better scalability to larger state spaces, making it the most practical choice for future problem extensions. Q-learning and MCTS both underperformed relative to their computational cost at this problem scale.

Under partial observability, QMDP demonstrated that knowledge of the opponent's unit counts has limited impact on win rate: performance degraded only modestly as observability decreased from perfect to fully blind. The more significant challenge proved to be opponent policy uncertainty: QMDP's win rate dropped from 40% to 15% under policy mismatch, while POMCP's online planning reduced this sensitivity.

Future work could increase state space complexity (higher troop and resource limits, additional unit types) to stress-test DQN's scalability advantage and further motivate POMCP over offline planners. Within the current formulation, double Q-learning could address the overestimation bias observed in DQN's value estimates.

Contributions

Jeremiah implemented the initial MDP environment, value iteration, Q-learning, and MCTS. He also implemented the POMDP extension and solvers.

Michael improved the MDP implementation and applied DQN, and implemented both the Markov Game and POSG extensions.

The authors grant permission for this report to be posted publicly.